

Name- Raj Ghule

Email- ghuleraj10@gmail.com

Domain- Artificial Intelligence and Machine Learning

Duration- 4 weeks

Batch start date- 25 may

Github-

<https://github.com/RajGhule-10/sumerix-global-ai-ml-internship-raj-ghule/tree/main>

TASK 3 REPORT

→ MODEL VALIDATION, OVERFITTING CONTROL & HYPERPARAMETER TUNING

Using California Housing Dataset

TITLE

Model Validation, Overfitting Control & Hyperparameter Tuning

INTRODUCTION

Machine learning models should not only achieve good accuracy but also generalize well on unseen data. Model validation techniques help evaluate a model's performance and identify issues such as overfitting. Hyperparameter tuning further improves model performance by selecting the most suitable model configuration.

In this project, the California Housing Dataset was used to perform model validation, analyze overfitting, apply cross-validation, and optimize a Decision Tree Regressor using GridSearchCV. The optimized model was then compared with baseline regression models using RMSE and R^2 Score.

OBJECTIVE

The objective of this project was to validate machine learning models, detect overfitting, perform cross-validation, optimize model hyperparameters using GridSearchCV, and compare model performance to identify the most reliable model for house price prediction.

TECHNOLOGIES USED

The following technologies and libraries were used in this project:

- Python
- Pandas
- NumPy
- Scikit-learn
- Jupyter Notebook

DATASET INFORMATION

The California Housing Dataset was obtained from Scikit-learn. It contains information about housing districts in California, including median income, house age, average rooms, population, occupancy, latitude, and longitude.

The target variable of the dataset is the median house value, which was predicted using multiple regression models after validation and optimization.

OVERFITTING ANALYSIS

6. Detecting Overfitting using Decision Tree

```
▶ ▾  
# Training Decision Tree model  
  
tree = DecisionTreeRegressor(random_state=42)  
  
tree.fit(X_train, y_train)  
  
# Predictions on training and testing data  
  
train_pred = tree.predict(X_train)  
test_pred = tree.predict(X_test)  
  
# Calculating RMSE  
  
train_rmse = np.sqrt(mean_squared_error(y_train, train_pred))  
test_rmse = np.sqrt(mean_squared_error(y_test, test_pred))  
  
print("Training RMSE :", train_rmse)  
print("Testing RMSE  :", test_rmse)  
[6] ✓ 0.0s Python  
... Training RMSE : 3.0302402915898627e-16  
Testing RMSE  : 0.7059501046231196
```

→ A Decision Tree Regressor was initially trained using the training dataset. Model performance was evaluated on both the training and testing datasets by comparing RMSE values.

A noticeable difference between training and testing performance indicates that the model may be overfitting the training data. This analysis helps determine whether the model generalizes well to unseen data.

CROSS VALIDATION

7. Cross Validation

```
# Performing 5-Fold Cross Validation

cv_scores = cross_val_score(
    tree,
    X_scaled,
    y,
    scoring="neg_root_mean_squared_error",
    cv=5
)

cv_rmse = -cv_scores.mean()

print("Cross Validation RMSE:", cv_rmse)
```

[7] ✓ 0.3s Python

... Cross Validation RMSE: 0.9029281160721967

→ To obtain a more reliable estimate of model performance, 5-Fold Cross Validation was performed using the entire dataset.

Cross-validation reduces dependency on a single train-test split by evaluating the model across multiple data partitions. This provides a more stable estimate of the model's performance.

HYPERPARAMETER TUNING

8. Hyperparameter Tuning using GridSearchCV

```
# Defining parameter grid

param_grid = {
    "max_depth": [3, 5, 7, 10],
    "min_samples_split": [2, 5, 10]
}

# Applying Grid Search

grid = GridSearchCV(
    DecisionTreeRegressor(random_state=42),
    param_grid,
    scoring="neg_root_mean_squared_error",
    cv=5
)

grid.fit(X_train, y_train)

print("Best Parameters:")
print(grid.best_params_)
```

[8] ✓ 1.6s Python

... Best Parameters:
{'max_depth': 10, 'min_samples_split': 10}

→ GridSearchCV was used to identify the best hyperparameter combination for the Decision Tree Regressor.

Different values of maximum tree depth and minimum samples required for splitting were tested. The parameter combination that produced the lowest validation error was selected as the optimized model.

MODEL COMPARISON

Model	RMSE	R2 Score	
0	Linear Regression	0.745581	0.575788
1	Ridge Regression	0.745554	0.575819
2	Tuned Decision Tree	0.645430	0.682099

→ The optimized Decision Tree model was compared with Linear Regression and Ridge Regression using RMSE and R^2 Score.

This comparison helps evaluate whether hyperparameter tuning improved the model's prediction capability compared to baseline regression models.

FINAL MODEL SELECTION

The optimized Decision Tree Regressor was selected as the final model because it achieved the best overall balance between prediction accuracy and generalization.

Reasons:

- Model optimized using GridSearchCV
- Performance validated using Cross Validation
- Reduced risk of overfitting
- Better overall predictive performance

CONCLUSION-

In this project, model validation and optimization techniques were successfully applied to improve machine learning performance.

Overfitting analysis, cross-validation, and GridSearchCV were used to evaluate and optimize a Decision Tree Regressor. The optimized model was then compared with baseline regression models using RMSE and R^2 Score.

This project provided practical experience in model validation, hyperparameter tuning, and performance evaluation, which are essential techniques for developing reliable machine learning models for real-world applications.